Ractors don't share memory, so they can only access memory that they own. Ractors communicate with each other by sending messages, which are then received and handled.

This is actually similar to how Elixir's concurrency model works. Let's explore that next.

Elixir's Concurrency Model

Elixir uses processes to achieve concurrency. Don't be fooled by the name because Elixir's processes are not operating system processes. Instead, these processes are lightweight virtual processes implemented by the BEAM. The BEAM's process implementation forms the foundation for all code execution.

Processes don't share memory with each other—they can only access their own memory. (We'll cover some exceptions to this later in this chapter.) Processes send and receive messages between themselves to coordinate work. Messages are stored in a process mailbox. This lines up closely with how Ractors in Ruby work, which makes sense because both the BEAM and Ractors are actor-based.

Actors are the foundation of concurrency in actor-based programming. Each actor receives messages (from other actors) and then processes each message. Actors can respond to messages, send messages to other actors, or execute code locally. This lines up exactly with how Elixir works.

In Elixir, a process executes work by taking a message out of its mailbox—it will always process messages in the order that it receives them. The message is then executed by the process, and this process repeats. Processes can execute infinitely—always waiting for a new message—or they can be set to only handle a fixed number of messages.

A process can only ever execute one message at a time. This means that there's no concurrency inside of a process. This is an often overlooked benefit of Elixir's concurrency model. In Elixir, you're in complete control over whether code runs in parallel or not.

Parallelism in Elixir

If there's no concurrency within a process, you may be wondering how the BEAM executes code in parallel. The magic comes from the ability to have tens of thousands of processes (or more!) executing concurrently with each other.